Data Pipelines

Knowledge Sharing Series

Hendrik & Jonathan

Reiner Lemoine Institut

2026-06-23

Knowledge Sharing Workshop Series

Most popular topics — weighted 2/3 students, 1/3 research staff

1. **ESM: Data Pipelines** *(today)*
2. Software: Code architecture
3. ESM: Scenarios
4. ESM: PyPSA
5. Software: Logging/Error handling
6. Software: Visualization
7. Software: Code project setup
8. Software: Code reviews
9. Web: REST APIs (FastAPI)
10. Software: Testing

Weighting: students have fewer workshop opportunities; survey shows $\approx 2\times$ higher average interest.

Full topic list: RLI Knowledge Sharing.xlsx

Data Pipelines

Agenda

Part I

- **Why data pipelines?** — the problem with cascaded scripts
- **ETL/ELT** — concepts and tools
- **Examples** from RLI projects
- **Software landscape** — which tool fits when?

Part II

- **Hands-on:** Snakemake

Code and instructions:
https://github.com/rl-institut/workshop/tree/master/knowledge_sharing/data_pipelines

Survey

A few quick questions — Mentimeter

Figure 1: QR for Mentimeter

The Problem: Cascaded Scripts

```
step_1_download.py → step_2_clean.py → step_3_aggregate.py →
... → step_n_plot.py
```

What breaks when this grows?

- Which order do I run them in?
- What happens if step 3 fails halfway through?
- Which output file is the current one?
- How do I re-run only the parts affected by a data update?

Fail stories 1/2

The silent overwrite

`step_3.py` reads from `output.csv` — written by `step_2.py`.

Someone renames `step_2`'s output to `output_v2.csv` to keep the old version.

Forgets to update `step_3`.

⇒ The pipeline runs. No error. No warning.

⇒ The analysis uses a 6-month-old file.

⇒ The bug surfaces two weeks later in a review.

Fail stories 2/2

The partial failure

A download loop fails at file 47 of 200.

Nobody notices — the aggregation step runs on 153 files.

The resulting map shows suspiciously low wind capacity in northern Germany.

⇒ Both fails have the same root cause:

Dependencies between steps are implicit, not declared.

What a Pipeline Tool Gives You

Explicit dependencies
Each step declares what it needs and what it produces. Missing inputs → stop immediately.

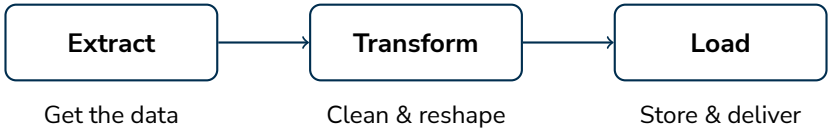
Partial re-execution
Output exists and is newer than input → skip. Changed input file → re-run only affected steps.

Auditability
Every run is logged: what ran, when, with what inputs.

Reproducibility
Same inputs → same outputs, always. The pipeline *is* the documentation.

ETL / ELT

Three stages of a data pipeline:



ELT (modern variant): Load raw data first, transform inside the database. Enabled by fast in-process engines like DuckDB.

Extract — Getting the Data

Common sources in energy research

- REST APIs: ENTSO-E, Marktstammdatenregister (MaStR), SMARD, OpenStreetMap
- File downloads: ZIP archives, CSV, Excel from government portals
- Databases: PostgreSQL, SQLite, remote object storage (S3)
- Geoserver: WMS, WFS, GeoJSON
- Web-Scraping: (if no API is present) web pages, chart data

Tools: `requests` Download to: Memory / Local files / Database

Common pitfalls

- Schema drift: a column gets renamed in the next API version

Transform — Reshaping the Data

Tools

- `pandas` — the standard; good for tables up to a few GB
- `polars` — faster pandas replacement; lazy evaluation
- `duckdb` — SQL directly on DataFrames and Parquet files; surprisingly fast

Typical operations in energy data

- Column mapping (rename, reorder, drop)
- Type casting (strings to `datetime`, `float` to `int`)
- Joining two datasets on a key (turbine ID, timestamp, spatial join)
- Resampling time series (hourly → daily, 15-min → hourly)
- Unit conversion, filtering outliers, filling gaps

Load — Storing the Results

Targets

- CSV — portable, human-readable, universally understood
- Parquet — columnar, compressed, typed; prefer for data > a few MB
- SQLite / PostgreSQL — structured queries, shared access
- Metadata JSON — record provenance: source URL, download date, version

Rule of thumb: store raw inputs untouched, transform into a separate `processed/` folder, deliver to `results/`. Never overwrite raw data.

ETL in Practice: A Code Sketch

```
import requests, pandas as pd

# Extract
r = requests.get("https://api.example.de/wind?year=2024")
df = pd.DataFrame(r.json())

# Transform
df["timestamp"] = pd.to_datetime(df["timestamp"])
df = df.set_index("timestamp").resample("D").sum()
df = df[df["power_kw"] > 0]          # drop zero-output rows

# Load
df.to_csv("results/wind_daily_2024.csv")
with open("results/metadata.json", "w") as f:
    json.dump({"source": r.url, "downloaded": str(date.today())}, f)
```

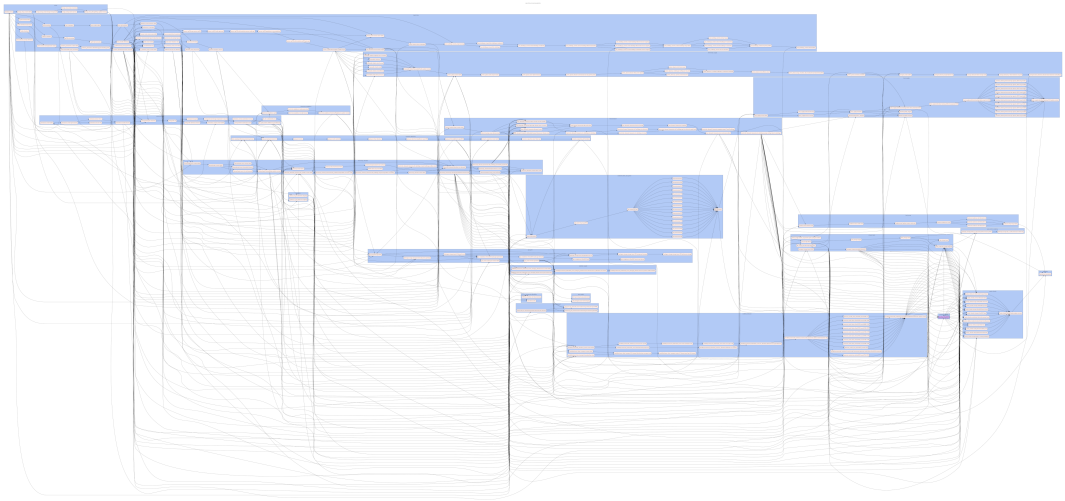
Project Examples

Project	Tool	Pipeline does	Why this tool?
eGon-data	Apache Airflow	Multi-dataset ingestion for grid- and energy system model	Scheduling, monitoring UI, team-shared
PV- und WFR	Snakemake	Geodata processing for RE potentials	File-based, Python-native, laptop-friendly
WKB	Node-RED	IoT sensor data ingestion & alerting	Visual editor, event-driven, no Python

All three solve the same problem — but in different contexts. The right tool depends on where and how the pipeline runs.

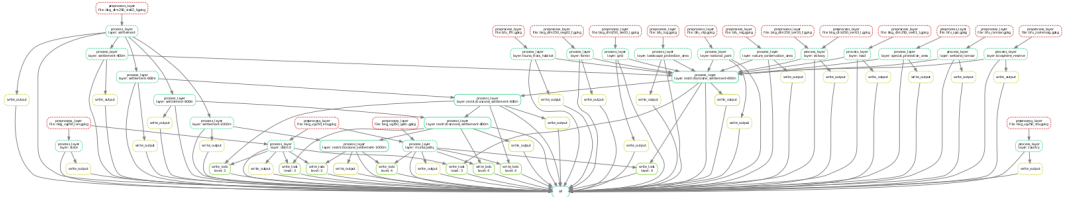
Project Examples 1/3: eGon-data

eGon-data pipeline for grid- and energy system model (Airflow)



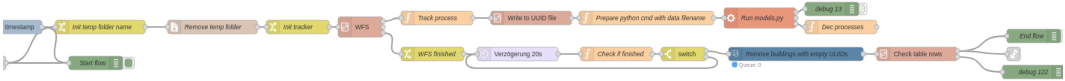
Project Examples 2/3: PV- und WFR

Geodata pipeline of PV- und Windflächenrechner (snakemake)



Project Examples 3/3: Berlin heat planning

Pipeline for building and heating data (NodeRed)



Software Landscape

Tool	Runs on	Interface
Makefile	local	code
doit	local	code
Snakemake	local → HPC/cloud	code
dlt	local → cloud	code
Prefect	local → server	code
Airflow	server	code
Node-RED	local → server	visual
n8n	local → server	visual

All code-based tools require Python/scripting knowledge. Visual tools (Node-RED, n8n) suit non-developers or event-driven workflows.

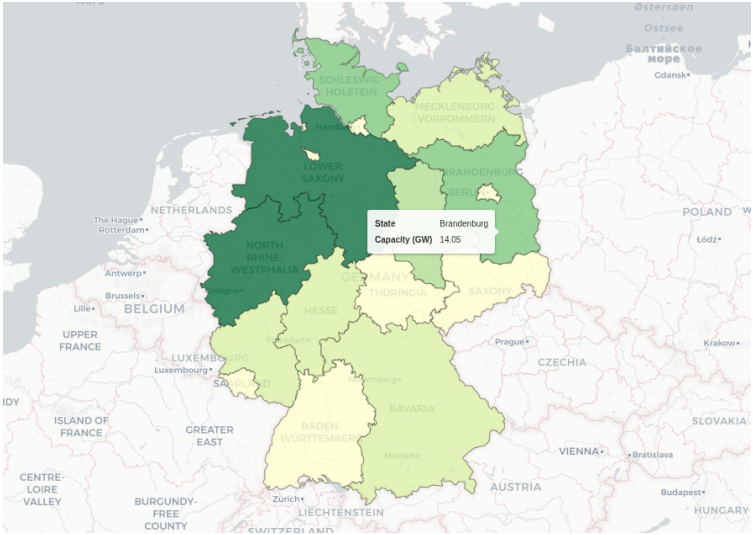
Which Tool for Which Job?

Question	Points toward
Runs on my laptop, no server?	Snakemake, doit, Makefile
Primarily file-based workflow?	Snakemake
Needs scheduling (daily, automated)?	Airflow, Prefect
Team is not Python-fluent?	n8n (visual), Node-RED
Mixed data + compute tasks, large team?	Prefect, Airflow
Minimal setup — `pip install` only?	Snakemake, dlt, doit

Questions & Discussion

Has anyone been in a situation where a script worked fine alone but caused problems when chained with others?

Part II: Snakemake hands-on



Hendrik & Jonathan
Tel:
E-Mail:
Web: www.reiner-lemoine-institut.de
Twitter: